



UAI301 Object Oriented Programming

Lab Assignment 5.1

Name: Amritanshu Mishra

Roll No.: 1025250008

Sub-Group: 2X11

Subject Code: UAI301

Subject Name: Object Oriented Programming

Question 1 Consider the Employee class with public name, protected `_department`, and private `salary`. **Create an object, modify `emp.name` and `emp._department`, and attempt `emp.salary = 70000`.** Print all three values, then use name mangling to print the original salary. Does `emp.__salary = 70000` modify the original private variable?

```
In [1]: class Employee:
        def __init__(self):
            self.name = "Ravi"
            self._department = "CSE"
            self.__salary = 50000

emp = Employee()

emp.name = "Amit"
emp._department = "ECE"
emp.__salary = 70000

print("name:", emp.name)
print("_department:", emp._department)
print("__salary:", emp.__salary)
print("Original private salary:", emp._Employee__salary)

print("\nAnswer: No, emp.__salary = 70000 does not modify the original pr
print("It creates a new attribute outside the class because of name mangl

name: Amit
_department: ECE
__salary: 70000
Original private salary: 50000

Answer: No, emp.__salary = 70000 does not modify the original private var
iable.
It creates a new attribute outside the class because of name mangling.
```

Question 2 Create a Patient class with public name, protected `_age`, and private **disease (accessed using name mangling)**. Display all three, and try to access disease directly to observe the resulting error.

```
In [2]: class Patient:
        def __init__(self, name, age, disease):
            self.name = name          # public
            self._age = age           # protected
            self.__disease = disease  # private

        p1 = Patient("Rahul Sharma", 45, "Diabetes")

        print("Patient Name:", p1.name)
        print("Patient Age:", p1._age)
        print("Patient Disease (via name mangling):", p1._Patient__disease)

        try:
            print(p1.__disease)
        except AttributeError as e:
            print("Error accessing __disease directly:", e)
```

```
Patient Name: Rahul Sharma
Patient Age: 45
Patient Disease (via name mangling): Diabetes
Error accessing __disease directly: 'Patient' object has no attribute
'__disease'
```

Question 3 Consider a class Employee with a private **salary = 50000**. Create an object, set `emp.salary = 80000` from outside the class, then print `emp.salary` and `emp._Employee_salary`. What is the output? Why are the two values different? Did `emp.__salary = 80000` change the original private variable?

```
In [3]: class Employee:
        def __init__(self):
            self.__salary = 50000

        emp = Employee()
        emp.__salary = 80000

        print(emp.__salary)
        print(emp._Employee__salary)

        print("\nExplanation:")
        print("emp.__salary prints 80000 because it is a new attribute.")
        print("emp._Employee__salary prints 50000 because it is the original priv")
        print("No, emp.__salary = 80000 did not change the original private varia")
```

80000
50000

Explanation:

emp.__salary prints 80000 because it is a new attribute.

emp._Employee__salary prints 50000 because it is the original private variable.

No, emp.__salary = 80000 did not change the original private variable.

Question 4 Create a DigitalWallet class with public owner_name, protected _wallet_type, and private **balance** and security_code. Implement deposit(amount), withdraw(amount, security_code), check_balance(), and change_security_code(). If the withdrawal amount exceeds the balance, display an appropriate message, and ensure balance/security code cannot be manipulated directly.

```
In [4]: class DigitalWallet:
    def __init__(self, owner_name, balance=0, security_code="0000"):
        self.owner_name = owner_name
        self._wallet_type = "Standard"
        self.__balance = balance
        self.__security_code = security_code

    def deposit(self, amount):
        self.__balance += amount
        print(f"Deposited {amount}. New Balance: {self.__balance}")

    def withdraw(self, amount, security_code):
        if security_code != self.__security_code:
            print("Incorrect security code. Withdrawal denied.")
            return
        if amount > self.__balance:
            print("Insufficient balance for withdrawal.")
        else:
            self.__balance -= amount
            print(f"Withdrew {amount}. New Balance: {self.__balance}")

    def check_balance(self):
        print("Current Balance:", self.__balance)

    def change_security_code(self, old_code, new_code):
        if old_code == self.__security_code:
            self.__security_code = new_code
            print("Security code changed successfully.")
        else:
            print("Old security code incorrect. Cannot change code.")

wallet = DigitalWallet("Aman", 5000, "1234")
wallet.deposit(2000)
wallet.withdraw(10000, "1234")
wallet.withdraw(3000, "1234")
wallet.check_balance()
```

```
wallet.change_security_code("1234", "5678")
```

Deposited 2000. New Balance: 7000
Insufficient balance for withdrawal.
Withdrew 3000. New Balance: 4000
Current Balance: 4000
Security code changed successfully.

Question 5 Design a SmartDoor class with public owner_name, protected _door_status, and private __security_pin. Implement lock_door(), unlock_door(pin), change_pin(old_pin, new_pin), and show_status(). The door starts Locked, unlocks only with the correct PIN, and the PIN can only be changed by providing the old PIN.

```
In [5]: class SmartDoor:
        def __init__(self, pin):
            self.owner_name = "Aman"
            self._door_status = "Locked"
            self.__security_pin = pin

        def lock_door(self):
            self._door_status = "Locked"
            print("Door is now Locked.")

        def unlock_door(self, pin):
            if pin == self.__security_pin:
                self._door_status = "Unlocked"
                print("Door Unlocked successfully.")
            else:
                print("Incorrect PIN. Door remains Locked.")

        def change_pin(self, old_pin, new_pin):
            if old_pin == self.__security_pin:
                self.__security_pin = new_pin
                print("PIN changed successfully.")
            else:
                print("Old PIN incorrect. PIN not changed.")

        def show_status(self):
            print("Door Status:", self._door_status)

door = SmartDoor("4321")
door.show_status()
door.unlock_door("0000")
door.unlock_door("4321")
door.show_status()
door.change_pin("4321", "1122")
door.lock_door()
door.show_status()
```

Door Status: Locked
Incorrect PIN. Door remains Locked.
Door Unlocked successfully.
Door Status: Unlocked
PIN changed successfully.
Door is now Locked.
Door Status: Locked

Question 6 Case Study 1: E-Commerce Order Processing (Protected Logistics). Create a parent class Order with protected `_shipping_status` = 'Processing' and a subclass ExpressOrder with `expedite_shipping()` and `track_order()` to demonstrate protected members in inheritance.

```
In [6]: class Order:
        def __init__(self):
            self._shipping_status = "Processing"

        class ExpressOrder(Order):
            def expedite_shipping(self):
                self._shipping_status = "In Transit - Priority"

            def track_order(self):
                return self._shipping_status

order1 = ExpressOrder()
print("Initial Status:", order1.track_order())
order1.expedite_shipping()
print("Updated Status:", order1.track_order())
```

Initial Status: Processing
Updated Status: In Transit - Priority

Question 7 Case Study 2: Ride-Hailing App (Taxi Driver Privacy). Create a Taxi class with public `driver`, protected `_location`, and private **contact**. Write **disp()** to show all three, demonstrate the error when accessing `contact` directly, and show admin access via name mangling (`_Taxi__contact`).

```
In [7]: class Taxi:
        def __init__(self, driver, location, contact):
            self.driver = driver
            self._location = location
            self.__contact = contact

        def disp(self):
            print(f"Driver: {self.driver}, Location: {self._location}, Contact: {self.__contact}")

taxi1 = Taxi("Suresh", "MG Road", "9876543210")

try:
    print(taxi1.__contact)
except AttributeError as e:
    print("Direct access error:", e)
```

```
taxi1.disp()
```

```
print("Admin access via name mangling:", taxi1._Taxi__contact)
```

Direct access error: 'Taxi' object has no attribute '__contact'

Driver: Suresh, Location: MG Road, Contact: 9876543210

Admin access via name mangling: 9876543210

Question 8 Case Study 3: Gaming Engine (Player Health & Invincibility).

Create a Player class with public username and private __hp (default 100).

Add take_damage(amount), heal(amount), and is_alive(), ensuring hp stays between 0 and 100, and simulate an attempted direct hp manipulation.

```
In [8]: class Player:
        def __init__(self, username):
            self.username = username
            self.__hp = 100

        def take_damage(self, amount):
            self.__hp = max(0, self.__hp - amount)
            print(f"{self.username} took {amount} damage. HP: {self.__hp}")

        def heal(self, amount):
            self.__hp = min(100, self.__hp + amount)
            print(f"{self.username} healed {amount} HP. HP: {self.__hp}")

        def is_alive(self):
            return self.__hp > 0

player = Player("Shadow")
player.take_damage(40)
player.heal(10)

player.__hp = 9999
print("Attempted direct HP change: player.__hp = 9999")
print("Is Alive:", player.is_alive())
print("Internal HP is still protected by name mangling.")
```

Shadow took 40 damage. HP: 60

Shadow healed 10 HP. HP: 70

Attempted direct HP change: player.__hp = 9999

Is Alive: True

Internal HP is still protected by name mangling.